

PROJECT REPORT

BudgetBuddy AI

An AI-powered personal expense tracker built with Firebase, FastAPI and Groq

Prepared by	Anushay Ayat
Institute	SMIT
Program	AI and Data Science
Date	October 2026
Technologies	HTML, CSS, JavaScript, Python, FastAPI, Firebase, Vercel, Groq

Contents

1. Introduction
 2. Requirements and Compliance
 3. System Architecture
 4. Technology Stack
 5. Detailed Design
 6. Implementation Highlights
 7. Testing
 8. Deployment
 9. Security Considerations
 10. Limitations and Future Work
 11. Conclusion
- Appendix A. Project Structure
- Appendix B. API Examples
- Appendix C. Troubleshooting

1. Introduction

1.1 Background

Managing day-to-day spending is a common challenge for students and young professionals. Paper notes and spreadsheets are easy to start but hard to keep up, and most finance apps either demand a long set-up or do not understand local habits such as paying through JazzCash or Easypaisa, receiving bank SMS alerts, or writing notes in Roman Urdu.

BudgetBuddy AI is a web application that makes expense tracking quick and conversational. A user signs in with Google, records expenses in a simple form, and can also describe a purchase in plain words ("Lunch 850 yesterday, paid by JazzCash") or paste a bank SMS and let an AI model fill in the details. The user can then ask questions about their own data, such as "Where did I spend the most this month?"

1.2 Problem Statement

Build an AI Expense Tracker that stores every expense in the cloud, shows all records and totals in a clear interface, supports deletion, uses a large language model through Groq, and is deployed with a public live link.

1.3 Objectives

- Provide a simple form to add an expense with title, amount, category (Food, Travel, Shopping, Bills, Other) and date, and let every user create their own custom categories.
- Persist all records in **Firebase Cloud Firestore**, separately for each signed-in user.
- Show saved expenses in a table below the form, with a delete option and live totals (number of expenses and total amount).
- Add AI features powered by **Groq**: quick add from text or SMS, question answering over the user's own data, category suggestion and a monthly review.
- Keep the AI key off the device by routing AI calls through a **Python FastAPI backend**.
- Deploy the frontend on **Firebase Hosting** and **Vercel**, and the backend on **Vercel**.

1.4 Scope

The core assignment covers expense entry, storage, display, deletion and totals. Beyond that, the project adds user-defined (custom) categories, budgets and category limits, charts, income and wallets, recurring bills, bill splitting, CSV and PDF export, an offline-first installable app (PWA) and an English / Urdu interface with right-to-left layout. These extras are optional for the assignment but make the application usable in daily life.

2. Requirements and Compliance

The table below maps each assignment requirement to its implementation.

Requirement	Implementation	Status
AI Expense Tracker using Groq	AI page: quick add, ask your spending, SMS parsing, category suggestion and monthly review via the FastAPI /api/ai endpoint	Done
Simple UI: Title, Amount, Category, Date, Add Expense button	Expense form in frontend/index.html with the five required categories plus user-created custom categories	Done
Store records in Firebase Firestore	Documents under users/{uid}/expenses, written with the Firebase Web SDK	Done
Show all saved expenses in a table below the form	Live table updated through a Firestore real-time listener	Done
Delete button to remove an expense	Delete with one-tap Undo	Done
Show total expenses and total amount	KPI cards and table footer update instantly	Done
Deploy and share the live link	Firebase Hosting and Vercel (see Section 8)	Done

2.1 Non-functional requirements

- **Privacy:** a user must never be able to read or write another user's data.
- **Secrecy:** no API key may be stored in the browser, in Firestore or in the repository.
- **Availability:** the app should remain usable without internet and sync later.
- **Usability:** responsive layout, dark mode, themes, and Urdu support.
- **Simplicity:** no frontend build step and a small, readable backend.

3. System Architecture

BudgetBuddy AI has three cooperating parts: a static frontend that runs in the browser, Firebase services for identity and data, and a small Python backend that talks to Groq on the user's behalf.

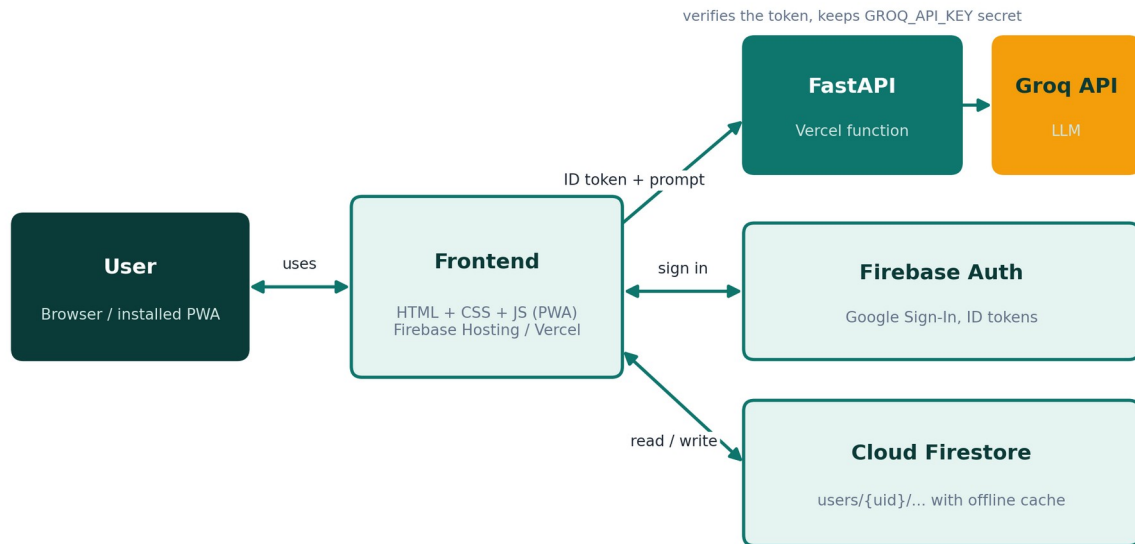


Figure 1: High-level architecture

3.1 Components

Component	Responsibility
Frontend (HTML, CSS, JS)	User interface, form handling, charts, i18n, offline behaviour, calls to Firestore and to the backend
Firebase Authentication	Google Sign-In; issues a signed ID token for each user
Cloud Firestore	Stores expenses, income, recurring rules, splits, budgets and custom categories per user; keeps an offline cache
FastAPI backend	Verifies the ID token, validates and rate-limits requests, calls Groq with the server-side key
Groq API	Runs the large language model (default openai/gpt-oss-20b)
Firebase Hosting / Vercel	Serve the static frontend over HTTPS; Vercel also runs the backend as a serverless function

3.2 Why a backend?

In the first version the user had to paste a personal Groq key into the browser. That works for a prototype but has clear weaknesses: the key sits in local storage, each user needs their own account, and a browser extension or script could read it. The FastAPI backend removes these problems. The key lives in a server environment variable, users only need to be signed in, and the server can enforce limits and input checks in one place.

3.3 Request flow for an AI call

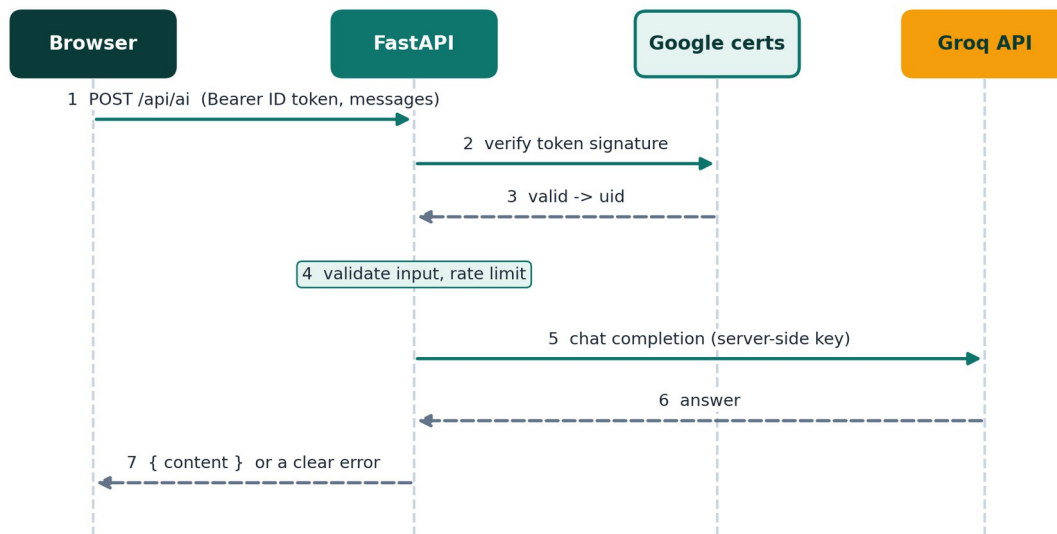


Figure 2: Sequence of an AI request

1. The browser reads the current user's Firebase ID token and sends it, with the messages, to POST /api/ai.
2. The backend verifies the token signature against Google's public certificates and extracts the user id.
3. The request body is validated (roles, number of messages, length) and checked against the per-user rate limit.
4. The backend calls the Groq chat completions endpoint with the secret key and the configured model.
5. The answer is returned as { "content": "..." }. Any failure is translated to a short readable message.

4. Technology Stack

Layer	Technology	Reason for choice
Frontend	HTML5, CSS3, vanilla JavaScript (ES modules)	No build step, fast loading, easy to read and deploy
Charts	Hand-written inline SVG	No library to download; works offline and in print
Backend	Python 3, FastAPI, Pydantic, httpx	Typed validation, async HTTP calls and automatic API docs
Token check	google-auth with cached public certificates	Official way to verify Firebase ID tokens without the Admin SDK
AI	Groq API, openai/gpt-oss-20b	Very low latency and a free tier suitable for a student project
Auth	Firebase Authentication (Google)	Secure sign-in with no password handling

Layer	Technology	Reason for choice
Database	Cloud Firestore	Real-time updates, offline persistence and per-user security rules
Offline / install	Service Worker and Web App Manifest	Installable app that works without internet
Hosting	Firebase Hosting, Vercel	Free HTTPS hosting; Vercel supports Python serverless functions
Testing	pytest, FastAPI TestClient, httpx MockTransport	Fast tests without real network calls

5. Detailed Design

5.1 Frontend

The frontend is a single-page application made of three files (index.html, style.css, app.js) plus small configuration modules. Pages are sections of one document that are shown or hidden, which keeps navigation instant.

Page	Purpose
Dashboard	KPI cards, six-month chart, category donut, budget status, category limits and category manager, forecast and insights
Expenses	Add / edit form (with a New category option), search, filters, sorting, table, totals and CSV export
Income and Wallets	Record income and see balances for Cash, Bank, JazzCash and Easypaisa
Recurring	Monthly bills that are created automatically
Split bills	Share a cost with friends and track repayments
AI	Connection status, quick add, ask your spending, monthly review

Every user-provided string is escaped before it is inserted into the page, so titles and notes cannot inject markup. The interface supports four colour themes, dark mode and a full English / Urdu switch with right-to-left layout.

5.2 Data model and storage

All data belongs to a user document, so a single security rule is enough to isolate users from each other.

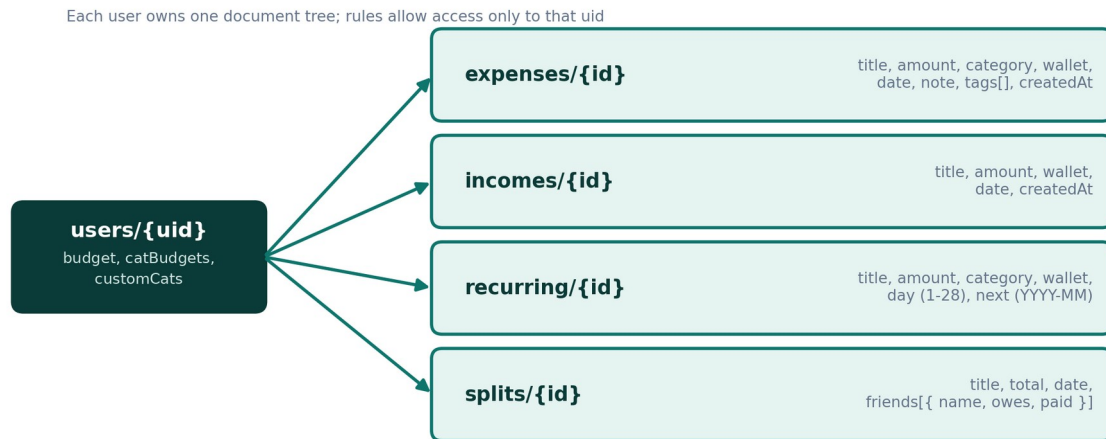


Figure 3: Firestore data model

```

rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /users/{uid}/{document=**} {
      allow read, write: if request.auth != null && request.auth.uid == uid;
    }
  }
}

```

User-defined categories are stored in a customCats array on the user document itself (a list of { name, color } objects). The existing security rule therefore already protects them and no rule change is needed. Each expense and each recurring rule keeps its category as plain text, so built-in and custom categories are treated in exactly the same way by the table, charts and reports.

Firestore is initialised with a persistent local cache and a multi-tab manager. Writes made while offline are queued by the SDK and sent automatically when the connection returns. The interface subscribes to the collections with real-time listeners, so the table and totals refresh as soon as data changes.

5.3 Authentication

Users sign in with the Google provider of Firebase Authentication. A popup is used by default, with a redirect fallback for browsers that block popups. After sign-in the app knows the user id (uid) and uses it as the root of every Firestore path. The same session produces an ID token that the backend verifies for AI calls.

5.4 AI module

The AI page offers four capabilities. All of them use one client function that posts to the backend, so adding a new feature only needs a new prompt.

Feature	How it works	Output
Quick add	Free text (English, Urdu or Roman Urdu) or a pasted bank / wallet SMS is converted to JSON	Pre-filled expense form that the user confirms

Feature	How it works	Output
Ask your spending	A compact JSON summary of the user's data is sent with the question	Short answer in the language of the question
Auto-category	Runs when a title is typed and the category has not been chosen manually	Suggested category among the allowed values (the five built-in ones and the user's own)
Monthly review	Summary of the current month	One overview line, two highlights, two suggestions

Prompt design

- **Structured output.** For quick add and category suggestion the backend uses JSON mode and a fixed list of allowed values (categories, wallets). The frontend still validates the result: the amount must be a positive number and unknown categories fall back to "Other".
- **Grounding.** Question answering is told to answer only from the JSON data and to say so when the data does not contain the answer.
- **Injection resistance.** Prompts state that expense titles and notes are plain data and never instructions.
- **Human in the loop.** AI never saves an expense by itself; it fills the form and the user presses Add.
- **Safety.** The prompts exclude investment and legal advice, and answers are short.

5.5 Backend API

The backend is one file (backend/main.py) of about 200 lines. Vercel runs it as a serverless function through a tiny entry point (api/index.py); locally it also serves the frontend so the whole app runs on one port.

Method and path	Auth	Purpose
GET /api/health	None	Reports status, whether the AI key is configured, whether auth is enforced and the model name
POST /api/ai	Firebase ID token	Chat completion through Groq; returns { content }

Validation and limits

- Between 1 and 6 messages; roles limited to system, user and assistant; each message at most 12,000 characters.
- Output limited to 700 tokens; temperature 0 in JSON mode and 0.4 otherwise.
- 30 requests per minute per user (in-memory sliding window).
- CORS restricted to the origins in ALLOWED_ORIGINS.

Error mapping

The browser never receives raw upstream errors. The backend converts them to a status code and a human-readable message that the frontend shows directly.

Situation	Status	Message (summary)
Missing or invalid Firebase token	401	Please sign in again
Invalid request body	422	Validation details
Rate limit or Groq free limit reached	429	Wait a minute and try again
Server key rejected, bad model name, other Groq error	502	AI service problem, contact the owner
Groq timeout (30 s)	504	The AI took too long
GROQ_API_KEY not configured	503	AI not configured on the server

5.6 Offline use and installation

A service worker caches the app shell (HTML, CSS, JavaScript, icon, manifest) and the Firebase SDK modules using a stale-while-revalidate strategy. Calls to /api/ are never cached, so AI answers and health checks are always live. A web app manifest makes the site installable on Android and desktop. An Offline badge is shown while the device is disconnected.

5.7 Internationalisation

All visible text goes through a translation function with an Urdu dictionary. Switching the language flips the page direction to right-to-left, loads the Noto Naskh Arabic font and re-renders every page. User-entered text is stored exactly as typed.

5.8 Custom categories

The five built-in categories (Food, Travel, Shopping, Bills, Other) satisfy the assignment, but real spending is more varied. Every user can therefore create categories of their own, with a name and a colour, and use them everywhere a category is used.

User flow

- **Create.** Choose New category in the Category dropdown of the expense form, the recurring dialog or the split-bill dialog; or press Manage categories next to the category limits on the dashboard.
- **Use.** The new category is saved at once, selected in the form that opened the dialog, and added to every category list in the app.
- **Delete.** In the category manager, Delete removes a custom category. If expenses or recurring bills still use it, the user is asked to confirm and they are moved to Other; the category limit is removed as well.

Rules enforced by the interface

Rule	Behaviour
Name length	1 to 24 characters; spaces are normalised.
Uniqueness	A name may not repeat an existing one, ignoring upper and lower case, and may not reuse a built-in name.
Allowed characters	The characters . / [] * ~ # \$ and the backtick are rejected, as are names that start with two underscores or match reserved object names.
Limit	At most 20 custom categories per user.
Order	Built-in categories first, custom categories next, and Other always last.
Colour	Chosen from a fixed palette; values that are not valid hex colours are replaced by the neutral grey of Other.

Integration with the rest of the app

- **Single source of truth.** The category list is rebuilt from one place whenever the stored list changes, which refreshes the form, recurring, split-bill and filter dropdowns together.
- **Charts and reports.** The donut chart, legend, table dots, category limits, CSV export and PDF report use the category colour and name without any extra code.
- **AI features.** The quick-add and auto-category prompts are built from the current category list, and an answer that is not in the list falls back to Other.
- **Synchronisation.** The list is read through the existing real-time listener on the user document, so a category created on one device appears on the others, and works offline like all other data.
- **Safety.** Category names are HTML-escaped wherever they are displayed, so a name cannot inject markup.

6. Implementation Highlights

- **Real-time table and totals.** A Firestore snapshot listener rebuilds the table and counters, so the totals are always consistent with the stored data.
- **Undo delete.** Deletion is applied immediately and a toast offers Undo, which writes the document back.
- **Forecast.** The month-end forecast combines the daily average so far with the recurring bills that are still due.
- **Recurring bills.** Rules store a day of the month and the next month to create. On app start any missing months are generated.
- **Reports.** CSV export builds the file in the browser; the PDF report uses a print stylesheet and the browser's Save as PDF.

- **Token verification without the Admin SDK.** The backend uses google-auth to verify the token against Google's public certificates, with the certificates cached through cachecontrol. This keeps the deployment small and avoids storing a service-account file.
- **Single code path for local and cloud.** The same FastAPI app runs under uvicorn locally (serving the frontend too) and as a Vercel function.
- **Custom categories.** Categories are saved as a { name, color } list on the user document and the dropdowns, chart colours and limits are all rebuilt from it. Deleting a category re-assigns its expenses to Other so no record is left without a valid category.

7. Testing

7.1 Automated tests

The backend has 17 automated tests written with pytest. They use FastAPI's TestClient and replace the Groq network call with httpx.MockTransport, and replace Firebase token verification with a stub. All 17 tests pass.

Area	What is verified
Health	Endpoint returns status, ai_configured and auth_required correctly
Successful call	Groq receives the Bearer key, the default model and temperature 0.4; the answer is returned as { content }
JSON mode	response_format is set and temperature is 0
Configuration	Missing GROQ_API_KEY returns 503
Upstream errors	Groq 429, 401, 404 and 500 are mapped to 429, 502, 502 and 502
Input validation	Empty list, unknown role, empty text, oversized text and too many messages return 422
Authentication	No header or wrong scheme returns 401; valid token passes; invalid token returns 401
Rate limiting	With a limit of three, the fourth and fifth calls return 429
CORS	Preflight request from an allowed origin is accepted with the correct header

```
$ python -m pytest -q
..... [100%]
17 passed
```

A smoke test also started the real server with uvicorn and confirmed that /api/health, the index page, the icon, the manifest and the configuration module are served, and that /api/ai returns 503 with a clear message when no key is set.

Frontend test for custom categories

The custom-category feature has its own automated test (tests/frontend/categories.test.js). It loads the real index.html and app.js in jsdom, replaces Firebase with a small in-memory mock and drives the interface through its events. It runs without network access, and all 29 checks pass.

Area	What is verified
Creation	Choosing New category opens the dialog; the category is saved to customCats, selected in the form, and added to the recurring, split and filter dropdowns; Other stays last.
Validation	Duplicate (any case), built-in name, dot, reserved word, leading underscores and empty name are rejected and nothing is saved.
Safety	A name containing HTML is escaped in the limits list, the manager and the dropdowns.
Use	An expense is saved with the custom category; the donut legend, the category limit row and the table colour show it; a limit can be saved.
Deletion	A category in use is deleted: its expenses move to Other, its limit is removed and it disappears from the dropdowns.
Start-up	Saved categories load on start; duplicates and invalid colours are cleaned; the Urdu label for New category is shown.

Command: npm install --no-save jsdom, then node tests/frontend/categories.test.js.

7.2 Manual test checklist

The following scenarios are recommended after each deployment to verify the complete system, because they depend on the live Firebase project and Groq key.

ID	Scenario	Expected result
T1	Open the site and click Continue with Google	Signed in; dashboard appears
T2	Add an expense with all fields	Row appears in the table; count and total increase
T3	Delete an expense, then press Undo	Row disappears, then returns
T4	Reload the page	Data is still present (stored in Firestore)
T5	Sign in with a second Google account	Sees none of the first account's data
T6	Go offline, add an expense, go online	Expense is saved locally, then synced
T7	AI page: type "Lunch 850 yesterday by JazzCash"	Form is pre-filled; user confirms to save
T8	AI page: ask "Where did I spend the most this month?"	Short answer based on the user's data
T9	Open /api/health	ai_configured is true and auth_required is true
T10	Call POST /api/ai without a token	401 response

ID	Scenario	Expected result
T11	Switch to Urdu	Text translated and layout right-to-left
T12	Export CSV and Report (PDF)	Files contain the selected period
T13	Expense form: choose New category, type Groceries, add it	Category is saved and selected; it appears in the recurring, split and filter dropdowns
T14	Add an expense in the new category and set a limit for it	Chart legend, table and category limit show the new name and colour
T15	Delete a category that has expenses (Manage categories)	Confirmation is shown; expenses move to Other; the category disappears on every device

8. Deployment

The frontend is a set of static files and can be served from either platform. The backend runs on Vercel as a Python serverless function. The recommended order is to deploy Vercel first, then Firebase Hosting.

8.1 Prerequisites

- A GitHub repository containing the project.
- A Firebase project with Google sign-in enabled, a Firestore database and the published security rules.
- A Groq API key from console.groq.com.

8.2 Vercel (frontend and backend)

1. Import the GitHub repository in Vercel with the preset Other and no build command.
2. vercel.json serves the frontend folder and rewrites /api/* to the FastAPI function in api/index.py; requirements.txt lists the Python packages.
3. Add the environment variables listed below and deploy.
4. Open /api/health on the new domain and check that ai_configured is true.
5. In Firebase Authentication, add the Vercel domain to the authorized domains so Google Sign-In works there.

8.3 Firebase Hosting (frontend)

1. Set API_BASE in frontend/api-config.js to the Vercel URL and push the change.
2. Add the web.app and firebaseapp.com addresses to ALLOWED_ORIGINS on Vercel and redeploy.
3. Run firebase login, firebase use --add and firebase deploy. firebase.json publishes the frontend folder.

8.4 Environment variables

Name	Required	Meaning
GROQ_API_KEY	Yes	Secret key for the Groq API
FIREBASE_PROJECT_ID	Yes (production)	Enables and configures Firebase token verification
ALLOWED_ORIGINS	Yes (production)	Comma-separated list of sites allowed to call the API
GROQ_MODEL	No	Overrides the default model name

Detailed commands for every step are in the README of the repository.

8.5 Live links

Platform	URL
Firebase Hosting	https://budgetbuddy-ai-23447.web.app
Vercel	https://budgetbuddy-ai-xi.vercel.app
GitHub repository	https://github.com/scientistayat-engineer/BudgetBuddy-AI

Replace the placeholders above with your real addresses after deployment.

9. Security Considerations

Risk	Mitigation
Reading another user's data	Firestore rules allow access only to users/{own uid}
API key theft	Key kept in a server environment variable; never sent to the browser or stored in the repository; .env is git-ignored
Abuse of the AI endpoint	Firebase ID token required, per-user rate limit, size and message limits
Cross-site calls	CORS limited to configured origins
Cross-site scripting	All user text is escaped before rendering; AI output is validated before use
Prompt injection through expense titles	Prompts treat titles and notes as data; AI cannot save or delete anything on its own
Exposed Firebase web config	These values are public identifiers by design; protection comes from rules and authorized domains
Privacy of AI questions	Only a compact summary of the user's data is sent, and only when the user asks a question
Malformed or malicious category names	Names are limited in length, count and characters, checked for duplicates and reserved words, and HTML-escaped when shown; colours must be valid hex values

10. Limitations and Future Work

10.1 Current limitations

- The rate limiter is in memory; on serverless hosting each instance counts separately.
- Recurring expenses are generated when the app opens, not by a scheduled server job.
- Voice input requires a browser with speech recognition (Chrome or Edge).
- Recurring rules cannot be edited yet and bill splitting is equal-share only.
- Backend behaviour and the custom-category logic are covered by automated tests; the rest of the frontend is verified manually.
- Custom categories cannot be renamed or re-coloured yet; to change one, delete it and create it again.

10.2 Future work

- Persistent rate limiting using Redis or Firestore.
- Scheduled generation of recurring expenses and daily reminder notifications.
- Receipt photo upload with automatic text extraction.
- Unequal bill splits and editable recurring rules.
- End-to-end frontend tests, beyond the custom-category test.
- Renaming and re-colouring custom categories, and choosing an icon for each.

11. Conclusion

BudgetBuddy AI meets every requirement of the assignment: expenses are entered through a simple form, stored in Firebase Firestore, listed in a table with live totals and can be deleted. The Groq-powered assistant turns natural language and SMS messages into structured expenses and answers questions about the user's own spending. Users can also define their own categories, so the tracker adapts to personal habits instead of forcing five fixed groups.

Moving the AI calls into a Python FastAPI backend improved the design in three ways: the API key is never exposed, only signed-in users can use the AI, and limits and validation are enforced in one place. The project is deployed on Firebase Hosting and Vercel, uses security rules for data isolation, works offline and supports both English and Urdu. It demonstrates how a small amount of backend code can safely connect a modern web frontend, a cloud database and a large language model.

Appendix A. Project Structure

```
budgetbuddy-ai/
  backend/      main.py (FastAPI app)
  frontend/    index.html, style.css, app.js, firebase-config.js,
               api-config.js, sw.js, manifest.webmanifest, images/
  api/         index.py (Vercel entry point)
  docs/        report, presentation, diagrams/
```

```
tests/          test_api.py, frontend/categories.test.js
firebase.json  firestore.rules  .firebaserc  vercel.json
requirements.txt  requirements-dev.txt  .env.example
LICENSE  README.md
```

Appendix B. API Examples

Health check

```
GET /api/health

{
  "status": "ok",
  "ai_configured": true,
  "auth_required": true,
  "model": "openai/gpt-oss-20b"
}
```

AI request

```
POST /api/ai
Authorization: Bearer <Firebase ID token>
Content-Type: application/json

{
  "messages": [
    { "role": "user", "content": "Give me 3 tips to save money" }
  ],
  "json_mode": false
}

200 OK
{ "content": "1. Set a weekly food budget ..." }
```

Appendix C. Troubleshooting

Symptom	Likely cause and fix
Google sign-in popup closes with an error on Vercel	The Vercel domain is missing from Firebase Authentication > Settings > Authorized domains
AI page says the server is not reachable	API_BASE is wrong, or the origin is missing from ALLOWED_ORIGINS (CORS). Redeploy after changing variables
AI page says there is no Groq key	GROQ_API_KEY is not set in Vercel, or the project was not redeployed afterwards
AI returns 401 after some time	The Firebase token expired; sign out and sign in again
Model error from Groq	Check the model name in GROQ_MODEL at console.groq.com/docs/models
Old version shown after deployment	The service worker cached the previous files; reload twice or clear site data
A new category does not show on another device	Categories sync through Firestore (users/{uid}.customCats). Check the connection, since offline changes sync later, and reload once